Exercise 12

#SKILL – 12: Full-Stack CRUD Application using React & Spring Boot

Prerequisites:

- Basic Understanding of Spring Boot
- Basic understanding of React
- Ability to integrate frontend & backend

A student management system needs full CRUD operations where the React frontend communicates with a Spring Boot backend through REST APIs. Users should be able to add, view, update, and delete student records through a simple UI, and all operations must update the data immediately without page reloads.

Your task is to build both the backend (Spring Boot) and frontend (React) parts and integrate them into a working full-stack application.

Tasks:

Backend – Spring Boot

1. Create CRUD REST endpoints in Spring Boot:

a. POST /students → Add a new student

b. GET /students → Retrieve all students

c. PUT /students/{id} → Update an existing student

d. DELETE /students/{id} → Delete a student by ID

e. Implement a Service layer and Repository layer.

f. Use @RestController, @PostMapping, @GetMapping, @PutMapping, @DeleteMapping.

g. Return responses using ResponseEntity.

Frontend – React

2. Create a React component StudentList:

a) Use axios to call GET /students.

b) Display all students in a table/list.

c) Include Delete and Update buttons for each row.

3. Create a React form AddStudent:

a) Use useState to capture form inputs (name, email, course).

b) Submit data to backend using axios POST.

c) Clear form after successful submission.

4. Implement Update and Delete features:

a. Delete: call axios DELETE and remove the student from UI.

b. Update:

i. Prefill form with selected student data

ii. Call axios PUT to update the record

5. Ensure the StudentList updates automatically after:

a) Adding a student

b) Updating a student

c) Deleting a student

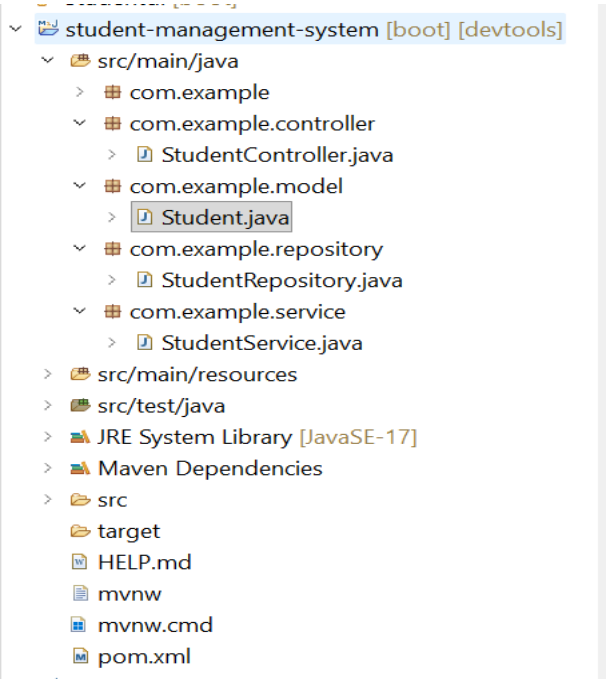
This can be done using state updates or refetching data after each operation.

6. Push the full-stack project into one GitHub repository. The repository must contain two folders with the following structure:

- frontend/ → React code
- backend/ → Spring Boot code

Step 1:

In SpringBoot, create the project



Student.java

```
package com.example.model;

import jakarta.persistence.*;

@Entity

public class Student {

    @Id

    @GeneratedValue(strategy = GenerationType.IDENTITY)

    private Long id;

    private String name;

    private String email;

    private String course;

    public Student() {}
```

```
    public Student(String name, String email, String course) {
        this.name = name;
        this.email = email;
        this.course = course;
    }
    public Long getId() { return id; }
    public String getName() { return name; }
    public String getEmail() { return email; }
    public String getCourse() { return course; }
    public void setName(String name) { this.name = name; }
    public void setEmail(String email) { this.email = email; }
    public void setCourse(String course) { this.course = course; }
}
```

StudentRepository.java:

```
package com.example.repository;
import org.springframework.data.jpa.repository.JpaRepository;
import com.example.model.Student;
public interface StudentRepository extends JpaRepository<Student, Long> {
}
```

StudentService.java:

```
package com.example.service;
import java.util.List;
import org.springframework.stereotype.Service;
import com.example.model.Student;
```

```
import com.example.repository.StudentRepository;
@Service
public class StudentService {
    private final StudentRepository repo;
    public StudentService(StudentRepository repo) {
        this.repo = repo;
    }
    public Student addStudent(Student student) {
        return repo.save(student);
    }
    public List<Student> getAllStudents() {
        return repo.findAll();
    }
    public Student updateStudent(Long id, Student newStudent) {
        Student student = repo.findById(id).orElseThrow();
        student.setName(newStudent.getName());
        student.setEmail(newStudent.getEmail());
        student.setCourse(newStudent.getCourse());
        return repo.save(student);
    }
    public void deleteStudent(Long id) {
        repo.deleteById(id);
    }
}
```

StudentController.java:

```
package com.example.controller;

import java.util.List;

import org.springframework.http.ResponseEntity;

import org.springframework.web.bind.annotation.*;

import com.example.model.Student;

import com.example.service.StudentService;

@RestController

@RequestMapping("/students")

@CrossOrigin(origins = "http://localhost:5173")

public class StudentController {

    private final StudentService service;

    public StudentController(StudentService service) {

        this.service = service;

    }

    @PostMapping

    public ResponseEntity<Student> addStudent(@RequestBody Student student) {

        return ResponseEntity.ok(service.addStudent(student));

    }

    @GetMapping

    public ResponseEntity<List<Student>> getAllStudents() {

        return ResponseEntity.ok(service.getAllStudents());

    }

    @PutMapping("/{id}")

    public ResponseEntity<Student> updateStudent(@PathVariable Long id,

                                                 @RequestBody Student student) {
```

```
        return ResponseEntity.ok(service.updateStudent(id, student));
    }
    @DeleteMapping("/{id}")
    public ResponseEntity<String> deleteStudent(@PathVariable Long id) {
        service.deleteStudent(id);
        return ResponseEntity.ok("Student deleted successfully");
    }
}
```

Application.properties:

```
spring.application.name=student-management-system
# Database Configuration
spring.datasource.url=jdbc:mysql://localhost:3306/hibernatedb
spring.datasource.username=root
spring.datasource.password=root@123
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
# Hibernate Configuration
spring.jpa.hibernate.ddl-auto=update
spring.jpa.show-sql=true
# Server Port (Optional)
server.port=8085
```

Step 2: Connect to MySQL Server and create Database.

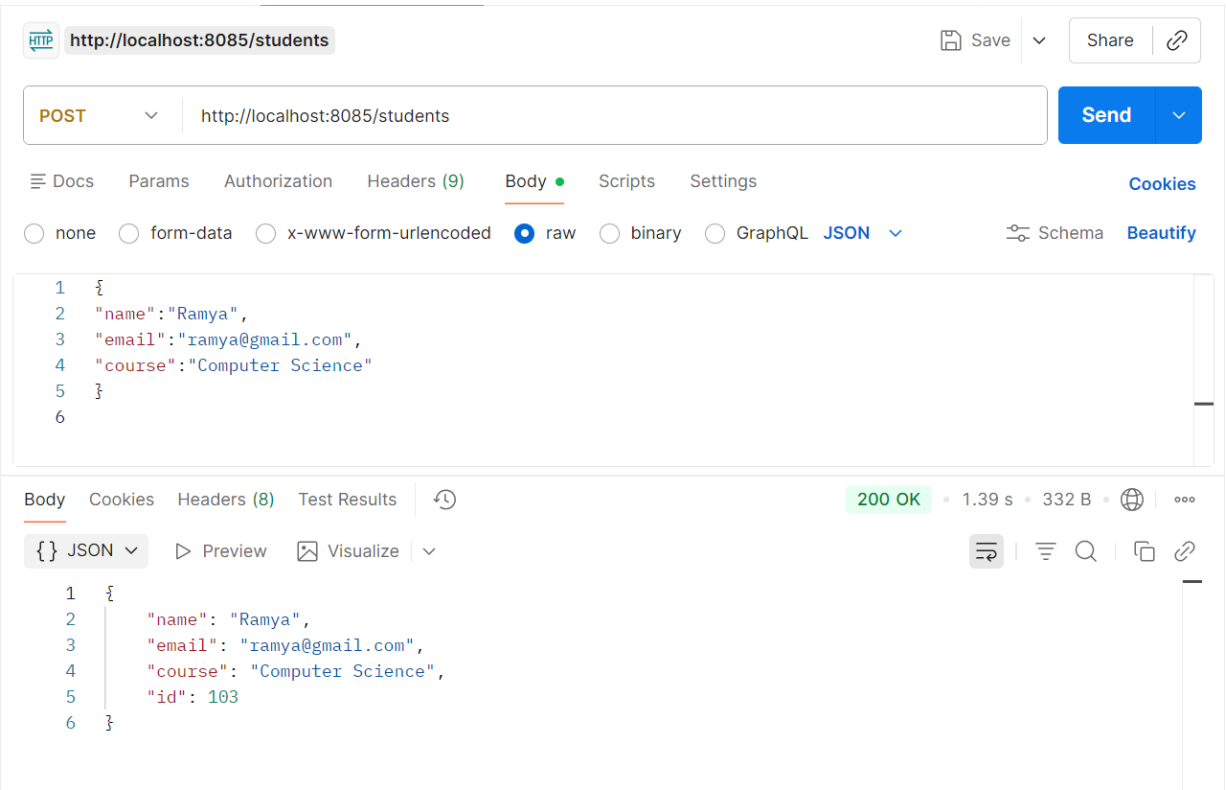
```
C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u root -p
Enter password: ********
ERROR 2003 (HY000): Can't connect to MySQL server on 'localhost:3306' (10061)

C:\Program Files\MySQL\MySQL Server 8.0\bin>mysql -u root -p
Enter password: ********
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 11
Server version: 8.0.45 MySQL Community Server - GPL

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
```

```
mysql> show databases;
+--------------------+
| Database           |
+--------------------+
| ecomm              |
| ecommerce          |
| hibernatedb        |
| imageupload        |
| information_schema |
| klh                |
| klu_db             |
| klu_role_based_auth |
| kludb              |
| kluh               |
| klujwtauth         |
| klujwtauth1        |
| librarydb          |
| mysql              |
| performance_schema |
| productdb          |
| sample             |
| sys                |
+--------------------+
18 rows in set (0.01 sec)

mysql> use hibernatedb;
Database changed
```

Step 3: Run the file StudentManagementSystemApllication.java

Step 4: In Postman, test all the endpoints.

Method	URL
POST	http://localhost:8085/students
GET	http://localhost:8085/students
PUT	http://localhost:8085/students/1
DELETE	http://localhost:8085/students/1

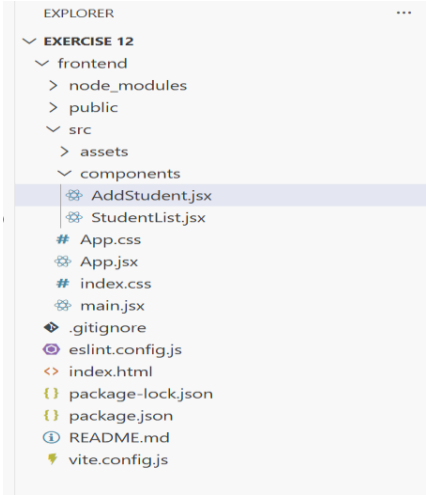


Step 5: In ReactJS,

Create Project

```
C:\Users\ADMIN\Desktop\Exercise 12>npm create vite@latest frontend
Need to install the following packages:
create-vite@9.0.2
Ok to proceed? (y) y


> npx
> create-vite frontend

│
◇  Select a framework:
│  React
│
◇  Select a variant:
│  JavaScript
│
```

```
C:\Users\ADMIN\Desktop\Exercise 12>cd frontend

C:\Users\ADMIN\Desktop\Exercise 12\frontend>npm install axios

added 23 packages, and audited 176 packages in 5s

42 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
```



AddStudent.jsx:

```
import { useState } from "react";
import axios from "axios";
function AddStudent({ refreshStudents, editingStudent, setEditingStudent }) {
  const [name, setName] = useState("");
  const [email, setEmail] = useState("");
  const [course, setCourse] = useState("");
  const handleSubmit = async (e) => {
    e.preventDefault();
    const student = { name, email, course };
    if (editingStudent) {
      await axios.put(`http://localhost:8085/students/${editingStudent.id}`, student);
      setEditingStudent(null);
    } else {
      await axios.post("http://localhost:8085/students", student);
    }
    setName("");
    setEmail("");
    setCourse("");
    refreshStudents();
  };
  return (
    <form onSubmit={handleSubmit}>
      <h3>{editingStudent ? "Update Student" : "Add Student"}</h3>
      <input
        placeholder="Name"
```

```
      value={name}
      onChange={(e) => setName(e.target.value)}
    />
    <input
      placeholder="Email"
      value={email}
      onChange={(e) => setEmail(e.target.value)}
    />
    <input
      placeholder="Course"
      value={course}
      onChange={(e) => setCourse(e.target.value)}
    />
    <button type="submit">
      {editingStudent ? "Update" : "Add"}
    </button>
  </form>
 );
}
export default AddStudent;
```

StudentList.jsx:

```
import { useEffect, useState } from "react";
import axios from "axios";
import AddStudent from "./AddStudent";
```

```
function StudentList() {
  const [students, setStudents] = useState([]);
  const [editingStudent, setEditingStudent] = useState(null);
  const fetchStudents = async () => {
    const res = await axios.get("http://localhost:8085/students");
    setStudents(res.data);
  };
  useEffect(() => {
    fetchStudents();
  }, []);
  const deleteStudent = async (id) => {
    await axios.delete(`http://localhost:8085/students/${id}`);
    fetchStudents();
  };
  return (
    <div>
      <AddStudent
        refreshStudents={fetchStudents}
        editingStudent={editingStudent}
        setEditingStudent={setEditingStudent}
      />
      <h2>Student List</h2>
      <table border="1">
        <thead>
          <tr>
            <th>Name</th>
```

```
            <th>Email</th>
            <th>Course</th>
            <th>Actions</th>
          </tr>
        </thead>
        <tbody>
          {students.map((s) => (
            <tr key={s.id}>
              <td>{s.name}</td>
              <td>{s.email}</td>
              <td>{s.course}</td>
              <td>
                <button onClick={() => setEditingStudent(s)}>
                  Update
                </button>
                <button onClick={() => deleteStudent(s.id)}>
                  Delete
                </button>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  );}
export default StudentList;
```

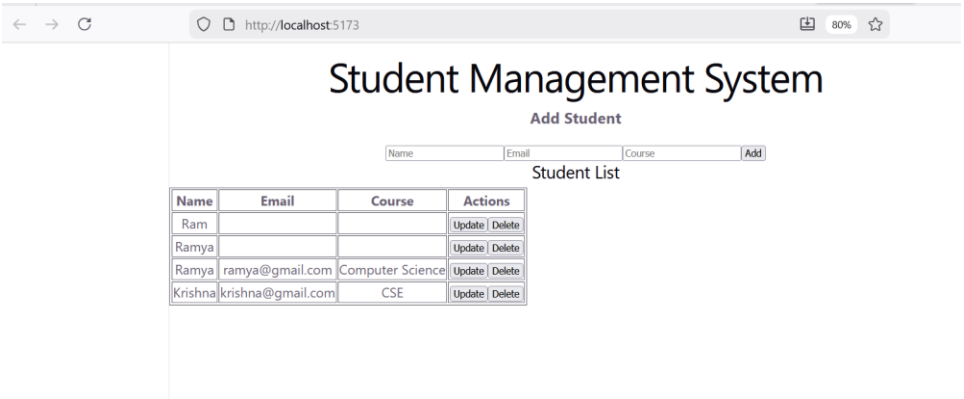
App.jsx:

```
import StudentList from "./components/StudentList";

function App() {

  return (

    <div>

      <h1>Student Management System</h1>

      <StudentList />

    </div>

  );

}

export default App;
```

Step 6:

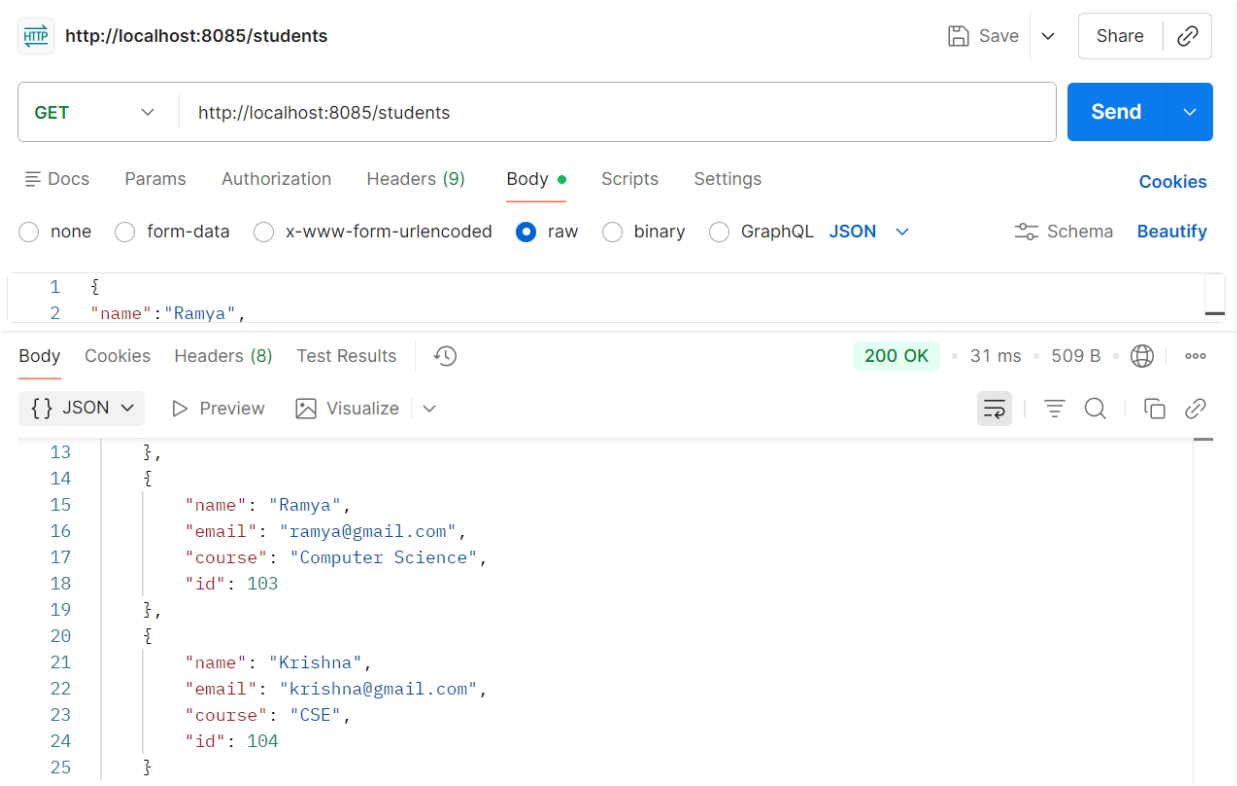
```
C:\Users\ADMIN\Desktop\Exercise 12\frontend>npm run dev

> frontend@0.0.0 dev
> vite


  VITE v8.0.0  ready in 412 ms

  ➜  Local:   http://localhost:5173/
  ➜  Network: use --host to expose
  ➜  press h + enter to show help
```

Output:



Step 7:

I have added new entry krishna, Check the updated list in Backend Postman again using Get



Likewise check all the options Add, Update, Delete in Frontend and check in Backend.

****